

Metanet 系统设计

本文档为 Metanet 设计文档体系的第二层：模块划分、接口定义、数据流。

文档体系：- [整体设计](#) — 两产品生态、三层架构、界面划分 - [概念设计](#) — 产品定位、核心理念、设计原则 - 系统设计（本文档） — 节点架构、合约、支付通道 - [详细设计](#) — 共识、挖矿、结算协议细节 - [测试设计](#) — 测试用例设计

一、三层架构

层	名称	职责	Token
Layer 1	BSV Main Chain	文件所有权 (Metanet DAG), HTLC 购买, x402 下载	BSV
Layer 2	Self-hosted Daemon	自托管文件服务, 现有 bitfs daemon	无 (自己的服务器)
Layer 3	Metanet Chain	去中心化 CDN, 存储合约, 支付通道	MNT Token

用户三种选择：

选择	适用场景	成本	可用性
仅上链 (BSV)	小文件, 永久存储	BSV 矿工费 (一次性)	区块链级别
自托管 (Daemon)	大文件, 完全控制	服务器成本 (持续)	取决于自己的基础设施
Metanet Chain 托管	大文件, 去中心化	MNT Token (持续)	CDN 级别 (多 Metanet Node 缓存)

三种方式可组合：元数据上链 + 热门内容 Metanet Chain 托管 + 冷门内容自托管。

两条链的职责划分：

	BSV Main Chain	Metanet Chain
共识	SHA256 PoW (原生)	SHA256 PoW (合并挖矿)
交易格式	Bitcoin 交易	完全相同
Script 引擎	Bitcoin Script	完全相同
原生代币	BSV	MNT Token
用途	文件所有权, 购买/出售	CDN 激励, 存储合约
面向用户	终端用户, Agent	Owner, Metanet Node
数据	Metanet 元数据	存储合约, 存储证明, 支付通道

二、Metanet Chain 基本设计

BSV 完全同构: - 相同的交易格式 (version, inputs, outputs, locktime) - 相同的 Script 引擎 (OP_CHECKSIG, OP_HASH256, OP_IF/ELSE, etc.) - 不同的创世块 (genesis block) - 不需要自定义交易类型 - 所有 “智能合约” 均为标准 Bitcoin Script

实现路径: fork BSV 节点软件, 最小修改: 1. 替换创世块 2. 调整区块参数 (大小、间隔) 3. 添加合并挖矿 (AuxPoW) 支持 4. 其他一切保持 BSV 原样

MNT Token: - 总供应量: 21,000,000 MNT (致敬 Bitcoin) - 初始区块奖励: 50 MNT - 减半周期: 每 210,000 块 - 区块时间: ~10 分钟 (与 BSV 同) - 最小单位: 1 satoshi = 0.00000001 MNT

合并挖矿 (Merged Mining): - SHA256 PoW, 可与 BTC/BSV 同时挖矿 - 矿工在 BTC/BSV coinbase 中嵌入 Metanet Chain block hash - Metanet Chain 验证 AuxPoW 即可确认区块 - 安全性随参与合并挖矿的算力增长

BSV 锚定: - 每 ~100 Metanet Chain 块, 将 Merkle root 写入 BSV 交易 - OP_RETURN <metanet_anchor_flag> <metanet_chain_merkle_root> <block_range> - 作用: 防止 Metanet Chain 长程攻击, 借助 BSV 的安全性

三、Metanet Node 经济模型

Metanet Node 三种收入:

收入来源	支付方	币种	触发条件
x402 检索费	用户/Agent	BSV	用户下载内容
CDN 托管费	Owner	MNT Token	Owner 签存储合约
挖矿奖励	协议	MNT Token	出块奖励

双币种分工: - BSV: 面向终端用户和 Agent, 用于 x402 微支付和 HTLC 购买 - MNT Token: 面向 Metanet Node 市场, 用于 CDN 托管费和挖矿奖励 - 普通用户不需要接触 Metanet Chain/Token — 只用 BSV 即可使用 BitFS - Token 需求 = Owner 对 CDN 服务的需求 (不是用户的需求)

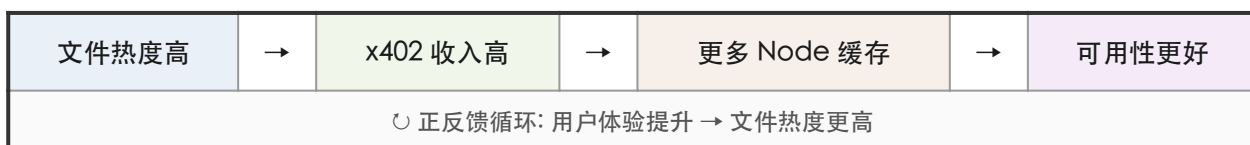
独立 `metanet` CLI:

```
bitfs daemon          # BitFS 自托管模式 (Layer 2)
metanet start          # Metanet Node 模式 (Layer 3, 对外提供 CDN 服务)
metanet start --mine   # Metanet Node + 矿工模式 (Layer 3, CDN + 挖矿)
```

`bitfs` 和 `metanet` 是独立二进制, 共享核心 Go 库。 `bitfs daemon` 面向文件拥有者, `metanet` 面向 CDN 节点运营者。

四、热数据: CDN 自组织模式

核心机制: Metanet Node 自发缓存热门内容, 利润驱动, 无需协议层管理。



特点: - 不需要存储合约 (Metanet Node 自愿缓存) - 不需要存储证明 (x402 交易记录本身证明 Metanet Node 有数据) - 不需要副本管理 (市场自动调节副本数) - 越热门的内容, 越多 Metanet Node 缓存, 类似传统 CDN 的缓存逻辑

Metanet Node 获取数据的方式: 1. Owner 主动推送: `bitfs put --store metanet` 上传到 Metanet Chain 2. Metanet Node 从 Owner daemon 拉取: 支付 x402 费用获取数据 3. Metanet Node 间批发: Node_A 从 Node_B 购买热门数据 (Token 支付通道)

Metanet Node 决策逻辑:

```
if 文件 x402_revenue > storage_cost + bandwidth_cost:
    缓存该文件 (利润驱动)
```

```
else:  
    不缓存（除非有存储合约）
```

五、冷数据: Archive 合约模式

核心机制: Owner 与 Metanet Node 签 1-to-1 存储合约, Owner 付 MNT Token。

存储合约生命周期:

1. Owner 选择 Metanet Node, 协商价格和期限
2. Owner 与 Metanet Node 签存储合约 (Bitcoin Script, Metanet Chain 交易)
3. Owner 用 ECDH 为该 Metanet Node 重新加密数据, 传输给 Metanet Node
4. Metanet Node 定期提交存储证明 (Merkle 挑战-响应)
5. 合约到期: Owner 可续期或让合约过期

存储证明: - ECDH 双层加密保证每个 Metanet Node 的副本独特 (无法互相抄袭) - 挑战确定性:
`challenge_k = SHA256(contract_txid || uint32_le(k))`, `k` 为期数 - 随机性来源于 `contract_txid` 在合约创建前不可预测 - 不使用区块哈希作为随机源 (避免矿工操纵) - 合约创建时 Owner 预计算所有 `N` 期的 `expected_proof_hash`, 编码到合约 UTXO 链中 - Metanet Node 定期提交存储证明: 根据 `challenge_k` 确定被挑战的数据块, 返回块数据 + Merkle proof - 验证者 (任何人) 可链上验证

副本策略: 协议不管副本策略 - Owner 自己决定冗余度

与 Filecoin 对比:

	Filecoin	Metanet Chain
副本独立性	PoRep (zk-SNARK)	ECDH 双层加密 (Method 42)
持续存储证明	PoS (zk-SNARK)	Merkle 挑战-响应
计算成本	GPU 密集, 数小时	毫秒级 ECDH + AES
副本管理	协议管理 (最少 <code>N</code> 副本)	Owner 自决 (签几份合约)
检索激励	薄弱 (检索矿工无激励)	强 (x402 直接收入)
代币用途	存储+检索+抵押	仅 CDN 托管+挖矿 (用户用 BSV)

六、内容分成

分成模式: Metanet Node 与 Owner 分享 x402 收入。

Owner 在 Metanet payload 中设置 `revenue_share` 字段:

```
revenue_share: {
  node_percent: 70,
  owner_percent: 30,
  min_price_per_kb: 1
}
```

两种合作模式:

模式	适用场景	Owner 付出	Owner 收入
分成模式 (热数据)	热门内容	无 (Metanet Node 自愿缓存)	x402 收入的 owner_percent
付费模式 (冷数据)	冷门内容	MNT Token (存储合约)	无 x402 收入 (或极少)

七、x402 支付通道

两种支付通道:

通道类型	方向	币种	用途
BSV 通道	User <-> Metanet Node	BSV	x402 流媒体微支付
Token 通道	Owner <-> Metanet Node	MNT Token	CDN 托管费持续支付
Token 通道	Metanet Node <-> Metanet Node	MNT Token	节点间数据批发

八、BSV <-> Metanet Chain 交互

文件发布 + CDN 托管流程:

1. Owner: bitfs put --store metanet myfile.txt
 - └ BSV 交易: 创建 Metanet 节点
 - └ Metanet Chain 交易: 创建 StorageDeal
 - └ 数据传输: ECDH 重加密 → 发送给 Metanet Node
2. Metanet Node 存储数据, 定期提交 StorageProof
3. User: bget bitfs://example.com/myfile.txt
 - └ 查询 BSV: 解析 Metanet 路径
 - └ 查询 Metanet Chain: StorageDeal → Metanet Node 列表
 - └ 请求 Metanet Node: x402 支付 → 获取加密数据

九、BRC 标准兼容

Metanet Chain 采用 BRC (Overlay 扩展):

BRC	名称	用途
BRC-31	Overlay Network	节点发现与路由
BRC-22	SHIP	提交交易到 Overlay
BRC-23	SLAP	查询 Overlay 服务
BRC-24	Topic Manager	管理 Overlay topic
BRC-25	Lookup Service	查询 Overlay 数据
BRC-87	Overlay Ads	广告可用服务
BRC-88	Overlay Tracking	追踪 Overlay 状态
BRC-64	Overlay Host	托管 Overlay 节点
BRC-103	Overlay Admin	管理 Overlay 节点
BRC-104	Overlay Sync	同步 Overlay 状态